МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

Отчёт по дисциплине «Управление знаниями и технологиями баз данных»

«Работа с метаданными и проектирование элементов BI системы»

Студент: _____
Осокин Н. А.

Преподаватель: _____
Попов С. Г.

«____» _____ 2026 г.

Санкт-Петербург, 2026 г.

Содержание

1 Постановка задачи 2

2 Базы-источники 4

2.1 База данных: authorities_dict (VM1, PostgreSQL) 4

2.2 База данных: authorities_ops (VM2, PostgreSQL) 6

2.3 База данных: authorities_archive (VM3, MariaDB) 8

3 Проектирование агрегирующей базы данных 11

3.1 Модель данных 11

3.1.1 Инстансы 11

3.1.2 Схемы и таблицы 12

3.1.3 Ограничения 12

3.1.4 Индексы 13

3.1.5 История запросов 13

3.1.6 Диаграмма 13

4 Архитектура системы 15

5 Реализация системы 18

5.1 Структура проекта 18

5.2 Сбор и сохранение метаданных 18

5.3 REST API 19

5.4 Федеративные запросы 19

6 Интерфейс системы 21

6.1 Общая структура главного окна 21

6.2 Работа с деревом схемы 22

6.3 Просмотр и сравнение данных 23

6.4 Сравнение схем 25

6.5 Конструктор федеративных запросов 25

1 Постановка задачи

Требуется создать систему, которая автоматически собирает описания структуры из нескольких независимых источников, сохраняет их в едином каталоге и предоставляет возможности для просмотра, сравнения и объединения данных. Система должна работать как связующее звено между источниками и не заменять их. Она не управляет бизнес-данными источников и не требует их остановки или перестройки. Основная ценность системы состоит в том, что она даёт единую, согласованную и актуальную картину описаний данных, а также позволяет получать объединённые результаты из разных источников.

Система должна подключаться к трём независимым источникам данных. Источники могут быть разнородными по своей природе и развиваться независимо друг от друга. Объём сведений в источниках значителен: количество записей в каждом из них должно быть не менее 200 000. Система должна автоматически собирать описания структуры данных, сохранять их в едином хранилище и вести историю изменений. Это необходимо для того, чтобы можно было не только увидеть текущее состояние описаний, но и сравнить его с предыдущими состояниями, а также сопоставить описания разных источников между собой.

В состав функций системы входит:

- автоматический сбор описаний структуры из всех подключённых источников;
- ведение единого каталога описаний и истории их изменений;
- просмотр и поиск описаний по источникам, сущностям и признакам;
- сравнение описаний двух источников или двух версий одного источника;
- соединение данных из разных источников по произвольным атрибутам;
- предоставление доступа к результатам через программный интерфейс.

Просмотр и поиск должны позволять быстро находить нужные сущности и их признаки, не обращаясь напрямую к каждому источнику. Сравнение должно показывать различия в составе сущностей, их признаков и связей, а также изменения, произошедшие между версиями. Объединение данных должно выполняться по произвольным общим признакам, даже если источники устроены по-разному. Результат объединения должен формироваться системой без необходимости переноса всех данных в одно хранилище и без изменения самих источников.

Система должна хранить описания структуры в упорядоченном виде, обеспечивающем целостность и однозначность связей между источниками, сущностями и признаками.

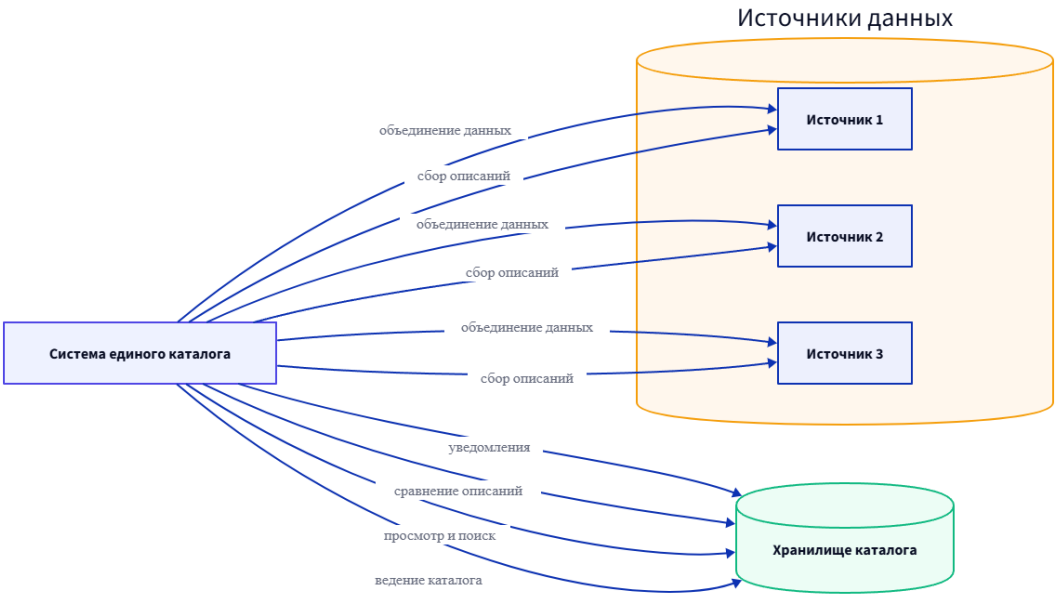


Рис. 1: Функциональная схема системы

2 Базы-источники

Система состоит из трёх независимых баз данных. Ниже описывается каждая из них: назначение, состав таблиц и ключевые атрибуты.

2.1 База данных: authorities_dict (VM1, PostgreSQL)

База данных справочников. Хранит медленно меняющиеся данные: виды деятельности, полномочия, базовые должности, лимиты полномочий по должностям, шаблоны доверенностей и падежные формы. Обновляется редко, читается часто — как прикладной логикой базы VM2, так и интерфейсом пользователя.

Схема включает семь таблиц. Таблица activity_kind хранит виды деятельности организации (name — название вида). Таблица authority представляет иерархический справочник полномочий: атрибут parent_id реализует самоссылку (дерево полномочий), level задаёт уровень иерархии (1 — группа, 2 — подгруппа, 3 — полномочие), text содержит полный текст полномочия.

Таблица basic_position содержит базовые должности. Таблица basic_position_limit задаёт лимиты полномочий для базовых должностей через составной первичный ключ (basic_position_id, authority_id). Таблица template хранит шаблоны доверенностей, а template_authority реализует связь M:N между шаблонами и полномочиями. Таблица padeg хранит падежные формы для ФИО, названий должностей и организаций.

На рисунке 2 показана ER-диаграмма базы данных authorities_dict.

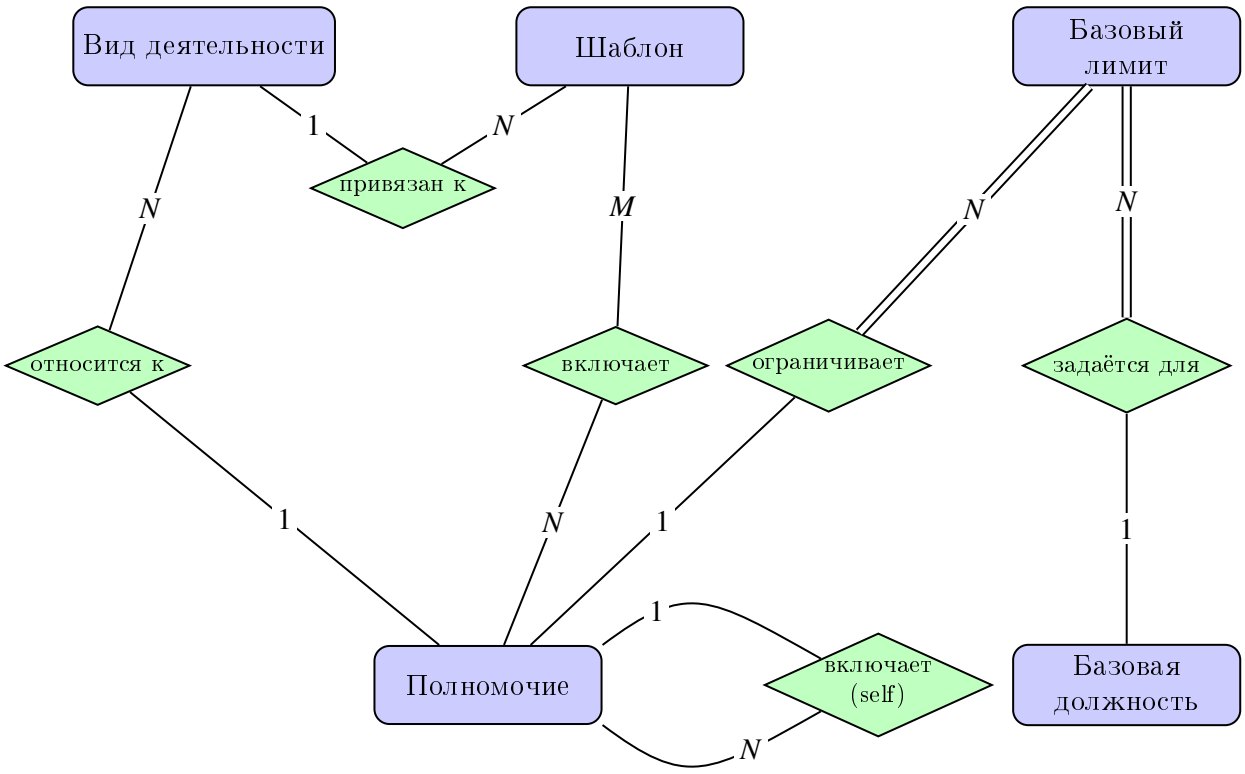


Рис. 2: ER-диаграмма БД №1 authorities_dict (VM1)

На рисунке 3 приведена схема базы данных, построенная по таблицам и связям.

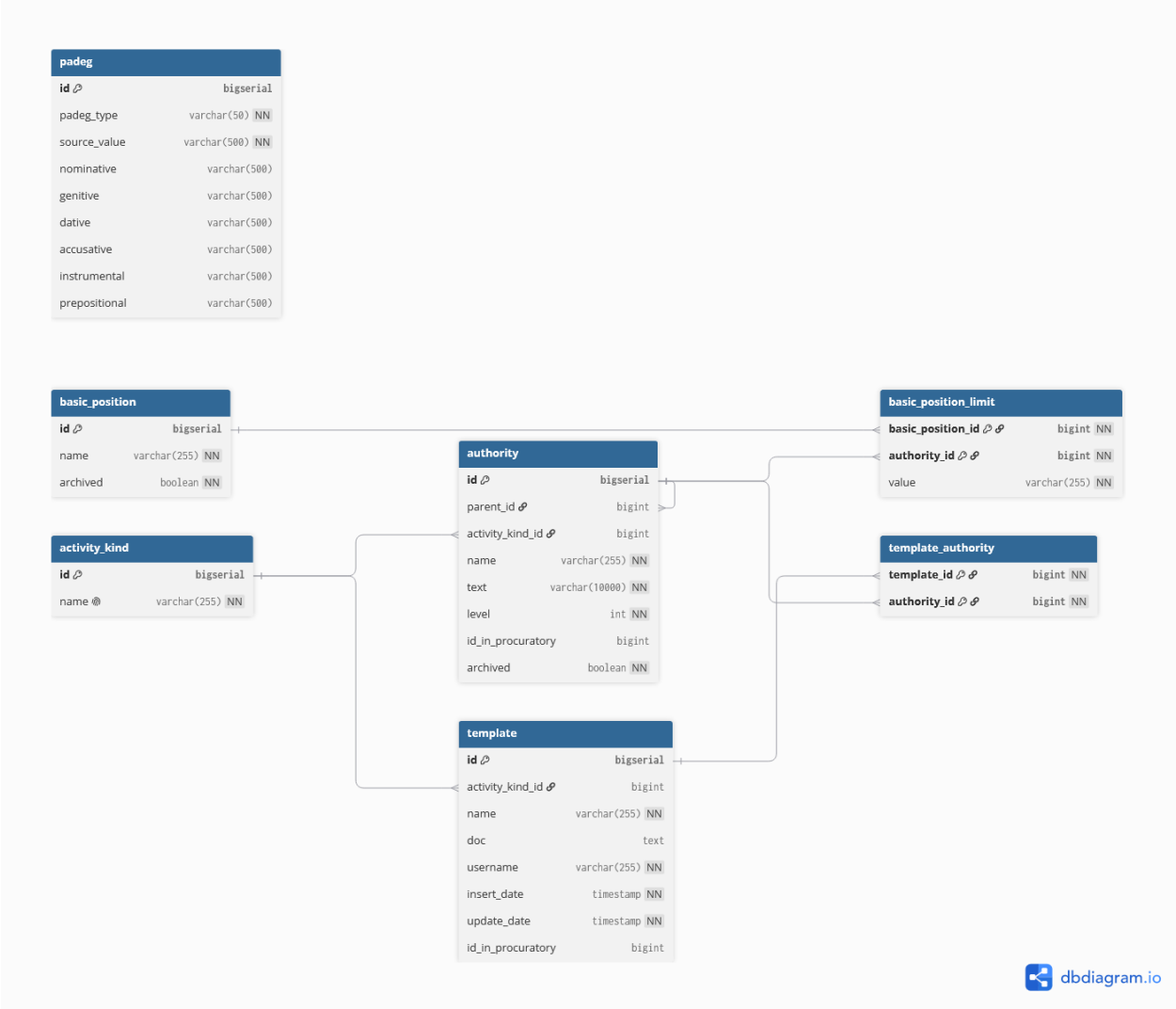


Рис. 3: Схема БД authorities_dict (VM1)

2.2 База данных: authorities_ops (VM2, PostgreSQL)

База данных операционного контура. Хранит организации, паспорта, должности, сотрудников, доверенности и связи доверенностей с полномочиями. Обновляется часто.

Схема включает шесть таблиц. Таблица organization — единый справочник организаций всех типов (type: ORG, JURIDICAL, MANAGEMENT). Таблица passport хранит паспортные данные. Таблица position — должности с cross-DB ссылками padeg_id и basic_position_id на VM1. Таблица employee — сотрудники, с cross-DB ссылкой fio_padeg_id на падежные формы.

Таблица procuration — центральная таблица фактов. Содержит номер, даты, основание, статус, ссылки на сотрудников и cross-DB ссылку

template_id на шаблон из VM1. Таблица procuration_authority реализует связь M:N между доверенностями и полномочиями с атрибутом limitation_value.

На рисунке 4 показана ER-диаграмма базы данных authorities_ops.

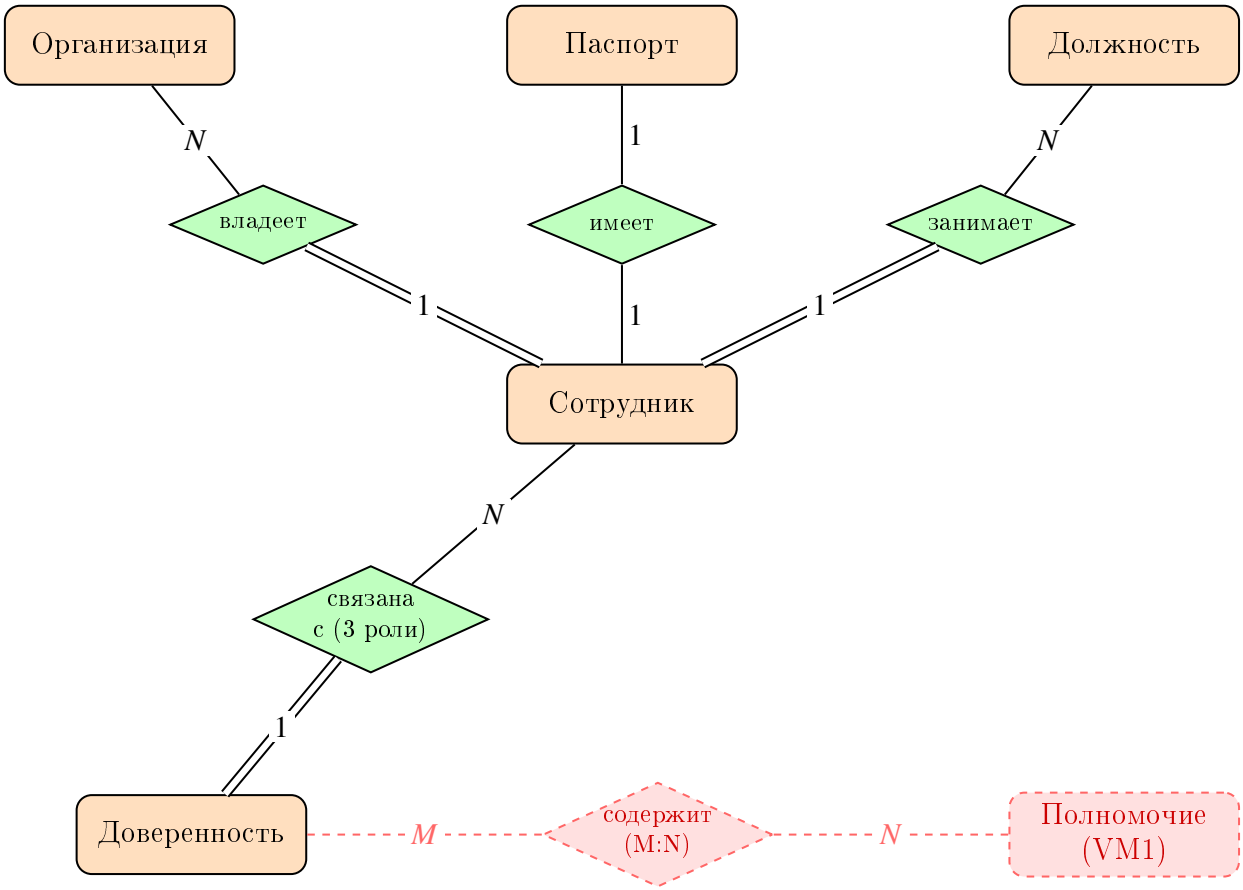


Рис. 4: ER-диаграмма БД №2 authorities_ops (VM2)

На рисунке 5 приведена схема базы данных, построенная по таблицам и связям.

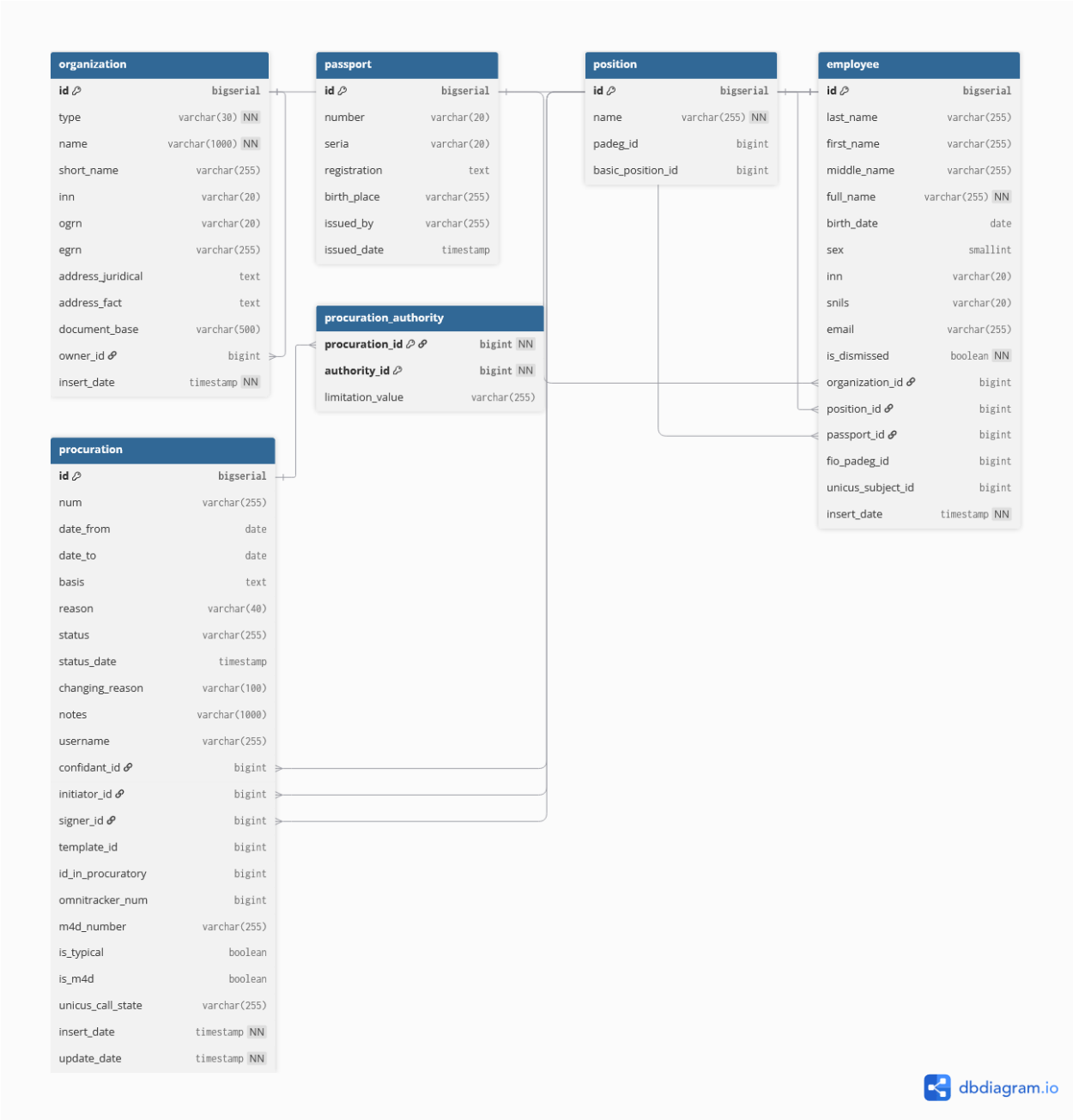


Рис. 5: Схема БД authorities_ops (VM2)

2.3 База данных: authorities_archive (VM3, MariaDB)

Архивная база данных на MariaDB. Хранит историю версий доверенностей, журнал аудита и снимки справочников.

Инстанс MariaDB содержит одну базу данных authorities_archive, в состав которой входят четыре таблицы. Таблица procuration_history реализует SCD Type 2 для доверенностей: при каждом изменении создаётся новая версия с полями valid_from и valid_to. Таблица audit_log хранит журнал действий пользователей над сущностями всех баз. Таблица

dict_authority_snapshot — SCD Type 2 снимки справочника полномочий из VM1. Таблица entity_mapping хранит соответствие идентификаторов между базами и внешними системами.

На рисунке 6 показана ER-диаграмма базы данных authorities_archive.

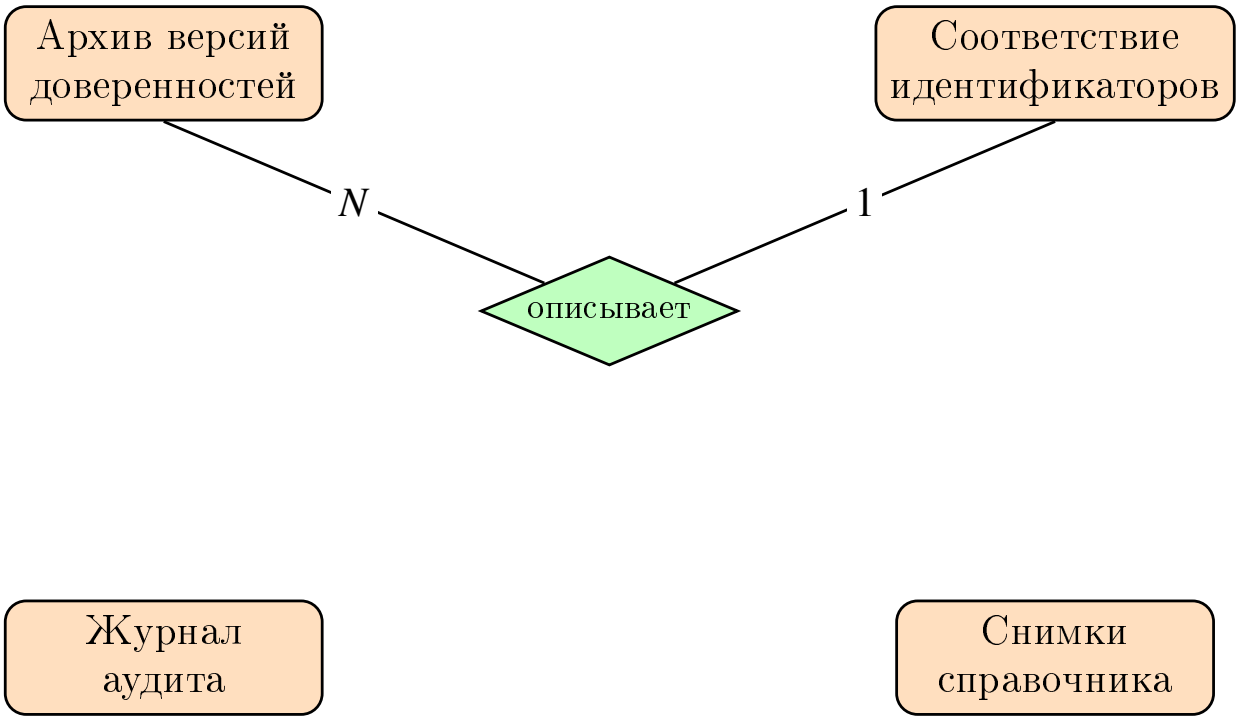


Рис. 6: ER-диаграмма БД №3 authorities_archive (VM3, MariaDB)

На рисунке 7 приведена схема базы данных, построенная по таблицам и связям.

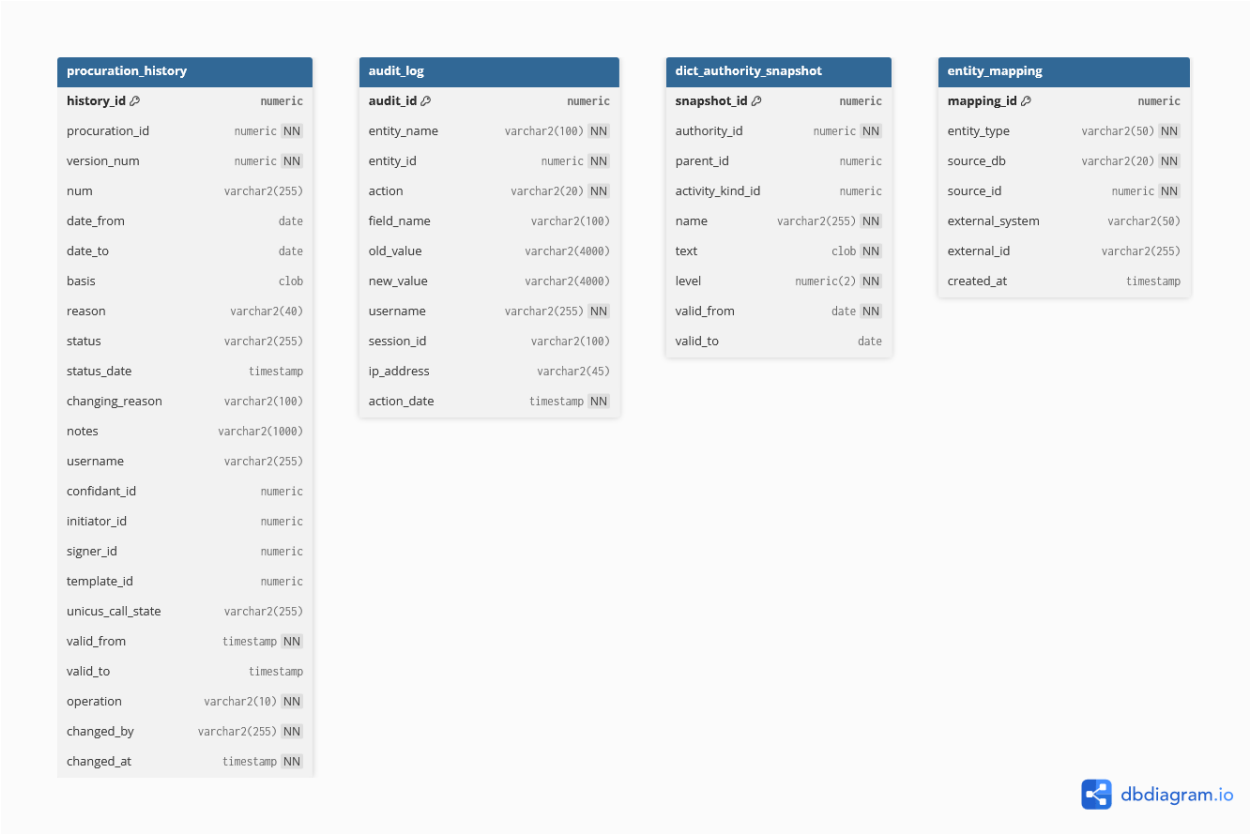


Рис. 7: Схема БД authorities_archive (VM3, MariaDB)

3 Проектирование агрегирующей базы данных

Агрегирующая база данных предназначена для хранения снапшотов схем всех источников. Она построена по нормализованной модели (третья нормальная форма). Ключи, ограничения и индексы, в том числе составные, хранятся реляционно: шапка объекта вынесена в одну таблицу, состав колонок — в дочернюю, с сохранением порядка следования. Это позволяет сравнивать описания структурно, а не по тексту, и корректно обрабатывать составные ключи любой длины.

3.1 Модель данных

Схема агрегирующей базы данных включает девять таблиц с общим префиксом sc_ (schema catalog). Иерархия объектов повторяет структуру самих СУБД: инстанс содержит схемы, схемы содержат таблицы, таблицы содержат колонки, ограничения и индексы. Ограничения и индексы, в свою очередь, имеют составные колонки, вынесенные в отдельные таблицы. Одна дополнительная таблица хранит историю федеративных запросов.

3.1.1 Инстансы

Таблица sc_instances описывает источники, к которым подключается сервис. Атрибуты: id — первичный ключ, label — человекочитаемое имя, connection_url — JDBC-URL без учётных данных (до 255 символов), username и password_encrypted — учётные данные, пароль хранится в зашифрованном виде через функцию pgp_sym_encrypt, alias — короткий уникальный идентификатор для федеративных запросов, kind — тип СУБД (POSTGRES, MARIADB, MYSQL, SQLITE, MSSQL, OTHER), created_at и updated_at — временные метки. Поле updated_at обновляется триггером sc_instances_set_updated_at при каждом изменении строки. На label, connection_url и alias наложены ограничения уникальности.

3.1.2 Схемы и таблицы

Таблица sc_schemas хранит схемы (namespace) внутри источника. Атрибуты: id, instance_id — внешний ключ на инстанс, schema_name, owner, comment. Пара (instance_id, schema_name) уникальна.

Таблица sc_tables описывает объекты, у которых есть колонки: обычные таблицы, партиционированные таблицы, представления, материализованные представления и foreign tables. Атрибут kind принимает одно из пяти значений. Пара (schema_id, table_name) уникальна.

Таблица sc_columns хранит колонки таблиц: имя, порядковый номер (ordinal_position), тип данных, признак is_nullable, значение по умолчанию (default_value), признак is_generated (GENERATED ALWAYS AS). Уникальность обеспечивается по двум парам: (table_id, column_name) и (table_id, ordinal_position).

3.1.3 Ограничения

Ограничения хранятся в двух таблицах. Таблица sc_constraints содержит шапку ограничения: id, table_id (таблица, к которой относится ограничение), constraint_name, type (PRIMARY_KEY, FOREIGN_KEY, UNIQUE, CHECK, EXCLUDE), expression (тело для CHECK и EXCLUDE), ref_table_id (целевая таблица для FOREIGN_KEY), on_delete и on_update (правила ссылочной целостности для FOREIGN_KEY: CASCADE, SET_NULL, SET_DEFAULT, RESTRICT, NO_ACTION).

Взаимоисключающие ограничения CHECK гарантируют корректность: ref_table_id заполнен только для FOREIGN_KEY, expression — только для CHECK и EXCLUDE, on_delete и on_update — только для FOREIGN_KEY.

Таблица sc_constraint_columns содержит состав ограничения: одна строка на каждую колонку, входящую в ключ. Атрибуты: constraint_id (внешний ключ на sc_constraints), column_name, position (порядок колонки в ключе, начиная с единицы), ref_column_name (для FOREIGN_KEY — имя колонки в целевой таблице, для остальных типов — NULL). Уникальность по (constraint_id, column_name) и (constraint_id, position) исключает дублирование и разрывы в порядке.

Такое разделение позволяет хранить составные ключи любой длины и запрашивать их структуру без разбора текста. Например, для составного первичного ключа таблицы procuration_authority вида PRIMARY KEY (procuration_id, authority_id) в таблице sc_constraints появится одна строка с типом PRIMARY_KEY, а в таблице sc_constraint_columns — две строки с позициями 1 и 2.

3.1.4 Индексы

Индексы устроены аналогично ограничениям. Таблица sc_indexes хранит шапку: index_name, is_unique, is_primary, kind (BTREE, HASH, GIN, GIST, SPGIST, BRIN) и definition — полный DDL, возвращаемый функцией pg_get_indexdef(), который необходим для выражений и частичных индексов. Таблица sc_index_columns хранит состав индекса с сохранением порядка: column_name, position.

3.1.5 История запросов

Таблица sc_query_history хранит журнал федеративных запросов: sql_text, status (OK, ERROR, TIMEOUT, CANCELLED), row_count, duration_ms, error, created_at. Эта таблица изолирована: у неё нет внешних ключей, потому что один федеративный запрос может обращаться к нескольким источникам одновременно. Снимки содержимого строк в агрегирующей базе не хранятся: сравнение строк выполняется по запросу на клиенте, поэтому отдельная таблица для них не нужна.

3.1.6 Диаграмма

Диаграмма агрегирующей базы данных приведена на рисунке 8.

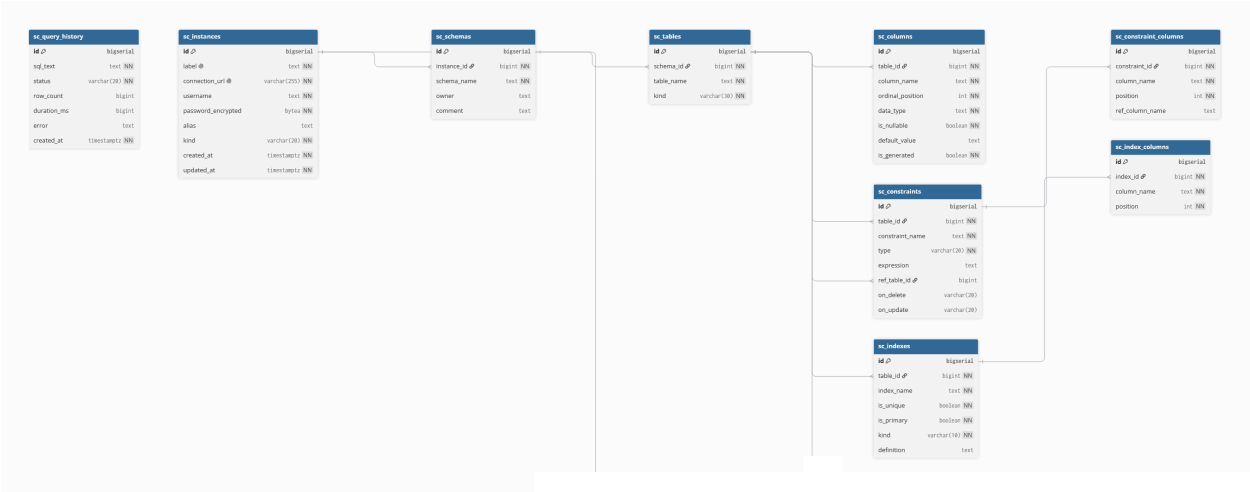


Рис. 8: Схема агрегирующей базы данных (метаданные)

4 Архитектура системы

Система построена по клиент-серверной схеме с выделенным сервисом сбора и хранения метаданных. Общая архитектура приведена на рисунке 9.

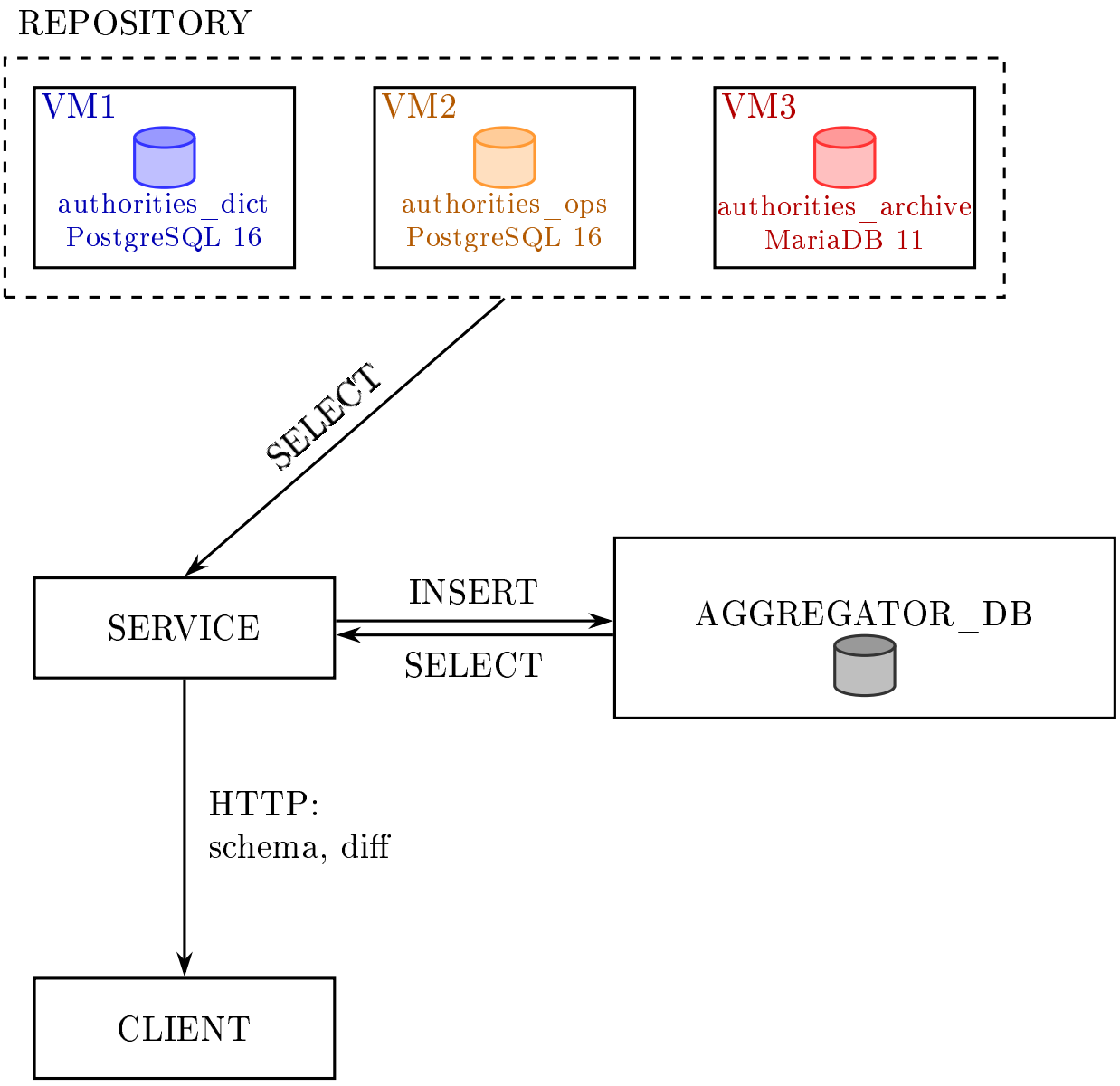


Рис. 9: Архитектура системы

В верхней части архитектуры расположен блок REPOSITORY, объединяющий три независимых источника данных. Каждый источник работает как отдельный контейнер с собственной изолированной сетью Docker, что исключает возможность прямого сетевого взаимодействия между ними и запрещает межбазовые SQL-запросы на уровне СУБД. Первый источник

(VM1) содержит базу authorities_dict на PostgreSQL 16 и хранит медленно меняющиеся справочные данные: виды деятельности, иерархический справочник полномочий, базовые должности, шаблоны доверенностей и падежные формы. Второй источник (VM2) содержит базу authorities_ops на PostgreSQL 16 и хранит операционные данные: организации, паспорта, сотрудников, должности и доверенности. Третий источник (VM3) содержит архивную базу authorities_archive на MariaDB 11 с историческими версиями доверенностей, журналом аудита и снимками справочников. Такое разделение источников по разным СУБД и разным контейнерам позволяет проверить, что система одинаково работает с разнородными источниками и не полагается на особенности конкретной реализации.

Центральным элементом архитектуры является сервис SERVICE, реализованный на Java 25 с использованием фреймворка Quarkus 3. Сервис подключается к каждому источнику по JDBC в режиме «только чтение» и собирает описания структуры данных: схемы, таблицы, колонки, ограничения и индексы. Соединение с источником открывается по требованию, читает метаданные одним пакетом запросов и сразу закрывается. Это гарантирует, что сервис не вмешивается в работу источника, не блокирует таблицы и не требует его остановки или перестройки. Сбор метаданных для трёх источников может выполняться параллельно, поскольку каждый источник работает в собственной сети и не зависит от остальных.

Собранные описания структуры сохраняются в агрегирующей базе данных AGGREGATOR_DB, работающей на PostgreSQL 17. Она хранит снапшоты схем всех источников в нормализованной форме: таблицы с префиксом sc_ повторяют иерархию самих СУБД (инстанс, схема, таблица, колонка, ограничение, индекс) и дополнительно содержат журнал федеративных запросов. Ключи, ограничения и индексы, в том числе составные, хранятся реляционно: шапка объекта вынесена в одну таблицу, состав колонок — в дочернюю, с сохранением порядка следования. Это позволяет сравнивать описания двух снапшотов структурно, а не по тексту, и корректно обрабатывать составные ключи любой длины.

Взаимодействие между сервисом и агрегирующей базой данных двунаправленное. Операция INSERT используется при сохранении нового снапшота: сервис записывает собранные описания в таблицы sc_* в рам-

ках одной транзакции, что исключает появление частично сохранённых снапшотов. Операция SELECT используется при чтении: когда сервису нужно отдать клиенту сохранённый каталог, сравнить две схемы или построить план федеративного запроса. Агрегирующая база данных не содержит бизнес-данных источников — только описания их структуры. Такое разделение позволяет хранилищу оставаться компактным и не зависеть от объёма данных в самих источниках.

Клиентская часть (CLIENT) взаимодействует с сервисом по HTTP. HTTP-запросы используются для получения списка источников, каталога схем, результата сравнения двух каталогов и выполнения федеративных SQL-запросов. Все операции возвращают полный JSON-ответ и завершаются сразу после отправки данных. После загрузки схемы клиент повторно запрашивает каталог, чтобы отобразить его актуальное состояние.

5 Реализация системы

5.1 Структура проекта

Исходный код сервера организован по слоям, приведённым в таблице 1.

Слой	Классы	Ответственность
controller	DbController, RowsController, JoinController	REST API
service	SchemaLoader, DiffService, RowsService, FederationService и др.	Сбор схемы, сравнение, выборка строк, федеративные запросы
repository	InstanceRepository, CatalogRepository, QueryHistoryRepository	Доступ к таблицам sc_*
model	Catalog, Schema, Table, Column, Constraint, Index	Объектная модель схемы и результатов
util	SqlQueries, CryptoUtil, RowsUtil	SQL-константы, шифрование

Таблица 1: Слои серверной части

Клиентская часть построена из компонентов: InstanceDock с формой подключения и деревом схем, SchemaTree, RowsExplorer и RowDiffView для работы с данными, SchemaDiffDrawer и DiffViewer для сравнения схем, JoinBuilder и QueryResultView для федеративных запросов. Состояние вынесено в runes-модули instances.svelte.ts и join.svelte.ts; модуль api.ts инкапсулирует обращения к REST.

5.2 Сбор и сохранение метаданных

Загрузка схемы начинается POST-запросом к ресурсу инстанса (/load). Обработчик проверяет существование инстанса, расшифровывает пароль и выбирает коннектор по типу СУБД. Коннектор PostgresConnector читает information_schema и pg_catalog, а MariaDbConnector — information_schema MariaDB. Оба коннектора работают через соединение «только чтение», поэтому сбор метаданных не блокирует таблицы источника и не требует его остановки.

Собранный каталог сохраняется в рамках транзакции в два этапа. Сначала записываются схемы, таблицы, колонки и индексы. Затем сохраняются ограничения: для внешнего ключа выполняется поиск целевой таблицы в уже сохранённом каталоге и заполняется ref_table_id. Если целевая таблица не найдена, внешний ключ пропускается с записью в журнал, чтобы не нарушать целостность каталога. После успешного сохранения каталог становится доступен для последующих запросов.

5.3 REST API

Все эндпоинты расположены под префиксом /api. Основные операции приведены в таблице 2. Запросы возвращают полный JSON-ответ и завершаются сразу после отправки данных.

Таблица 2: Основные эндпоинты REST API

Метод	Путь	Назначение
GET	/api/instances	Список подключённых источников
GET	/api/instances/{id}	Источник по идентификатору
POST	/api/instances	Создание источника
PATCH	/api/instances/{id}/password	Смена пароля
PATCH	/api/instances/{id}/alias	Смена alias для федерации
POST	/api/instances/aliases/backfill	Проставить alias существующим источникам
POST	/api/instances/{id}/load	Собрать и сохранить схему
POST	/api/instances/{id}/check	Сравнить сохранённую схему с текущей
GET	/api/instances/{id}/schema	Каталог источника
POST	/api/instances/compare	Сравнить две схемы
GET	/api/instances/{id}/tables/{schema}/{table}/rows	Страница строк таблицы
POST	/api/join	Выполнить федеративный запрос
GET	/api/join/history	История федеративных запросов

5.4 Федеративные запросы

Федеративный запрос формируется клиентом в виде структуры {tables, select, maxRows, offset, timeoutMs} и отправляется в POST/api/join. Сервер компилирует структуру в SQL, разбирает его через JSqlParser, определяет источники и нужные колонки, а затем выполняет чтение подтаблиц по JDBC. Прочитанные строки загружаются во временные

таблицы хранилища PostgreSQL, где и выполняется соединение. Такой подход позволяет соединять данные разных СУБД без переноса всех данных в одно постоянное хранилище: материализуются только строки и колонки, необходимые для конкретного запроса.

Для ограничения нагрузки на источники и хранилище установлены лимиты: до 1000 строк в ответе по умолчанию и до 100 000 максимально, время выполнения до 30 с по умолчанию и до 300 с максимально, до 500 000 строк из одного источника. Результат истории сохраняется в таблице sc_query_history.

6 Интерфейс системы

6.1 Общая структура главного окна

Интерфейс выполнен в виде одностраничного приложения. Все элементы расположены в одном окне и не требуют перехода между отдельными страницами. Главное окно разделено на левую панель источников и рабочую область. Такое расположение позволяет одновременно видеть список подключённых баз, структуру выбранной таблицы и результат её просмотра или сравнения.

Структура главного окна показана на рисунке 10. В левой части находится форма подключения нового источника, список загруженных инстансов и дерево «инстанс — схема — таблица». В правой части расположены переключатель режимов «Данные» и «Джойн», рабочая область и выдвижная панель сравнения схем.

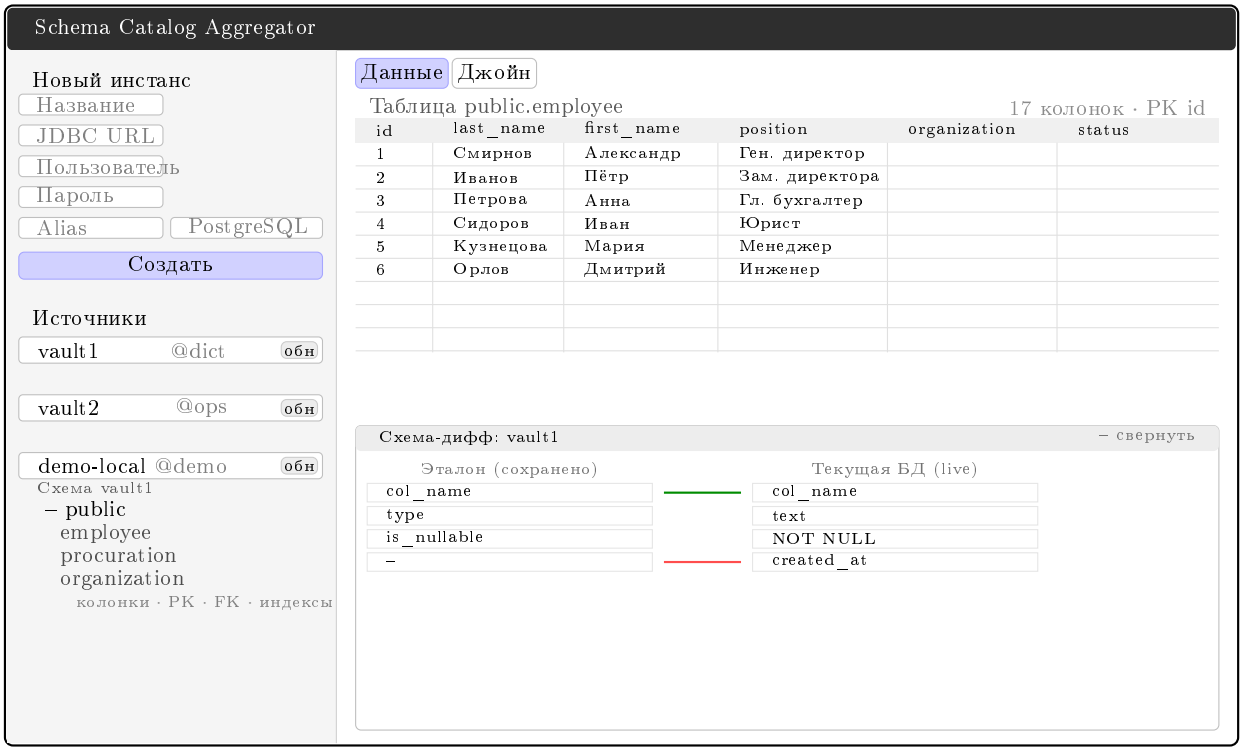


Рис. 10: Структура главного окна

Для каждого инстанса доступна кнопка обновления схемы. Форма подключения содержит название источника, JDBC-URL, имя пользователя, пароль, необязательный alias и тип СУБД. После создания источник

появляется в общем списке, а его схема загружается в дерево. На рисунке 11 показано главное окно с формой подключения и списком источников.



Рис. 11: Главное окно: подключение источника и список инстансов

6.2 Работа с деревом схемы

Дерево схемы отображает иерархию объектов источника. При раскрытии таблицы показываются её колонки, ограничения и индексы. Для каждой колонки указаны тип, признак NOT NULL, значение по умолчанию и признак генерации. Ограничения отображаются с составом ключа и правилами ON DELETE/ON UPDATE; индексы — с типом и перечнем колонок. Это позволяет изучить структуру таблицы, не открывая отдельный клиент СУБД.

При выборе таблицы в дереве в рабочей области открывается панель данных (рисунок 12). Выбранная таблица выделяется в дереве, а рядом с ней показывается число колонок, ограничений и индексов.

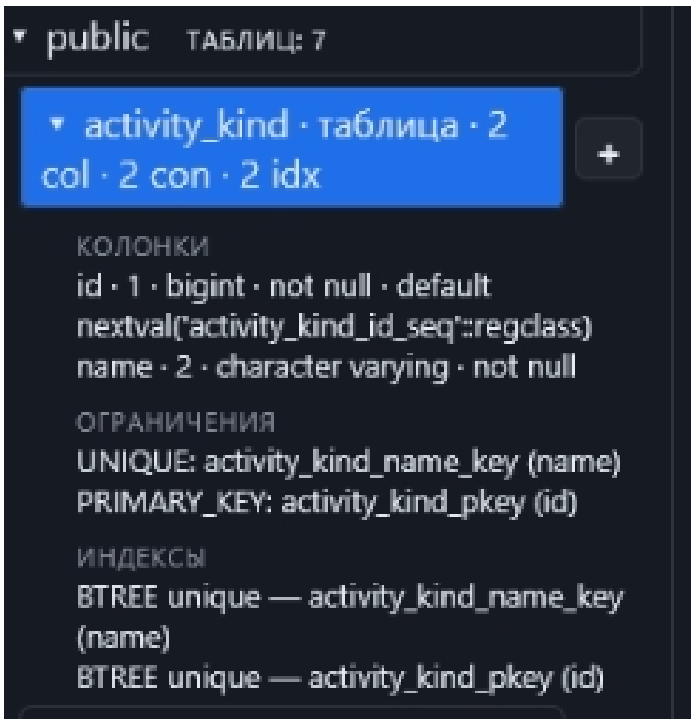


Рис. 12: Дерево схемы: колонки, ограничения и индексы

6.3 Просмотр и сравнение данных

Панель данных выполняет постраничную выборку строк. Строки упорядочиваются по первичному ключу, что обеспечивает детерминированную пагинацию; размер страницы по умолчанию равен 100, максимальный — 200. В заголовке панели указываются имя таблицы, число колонок и состав первичного ключа. На рисунке 13 приведён просмотр строк таблицы.

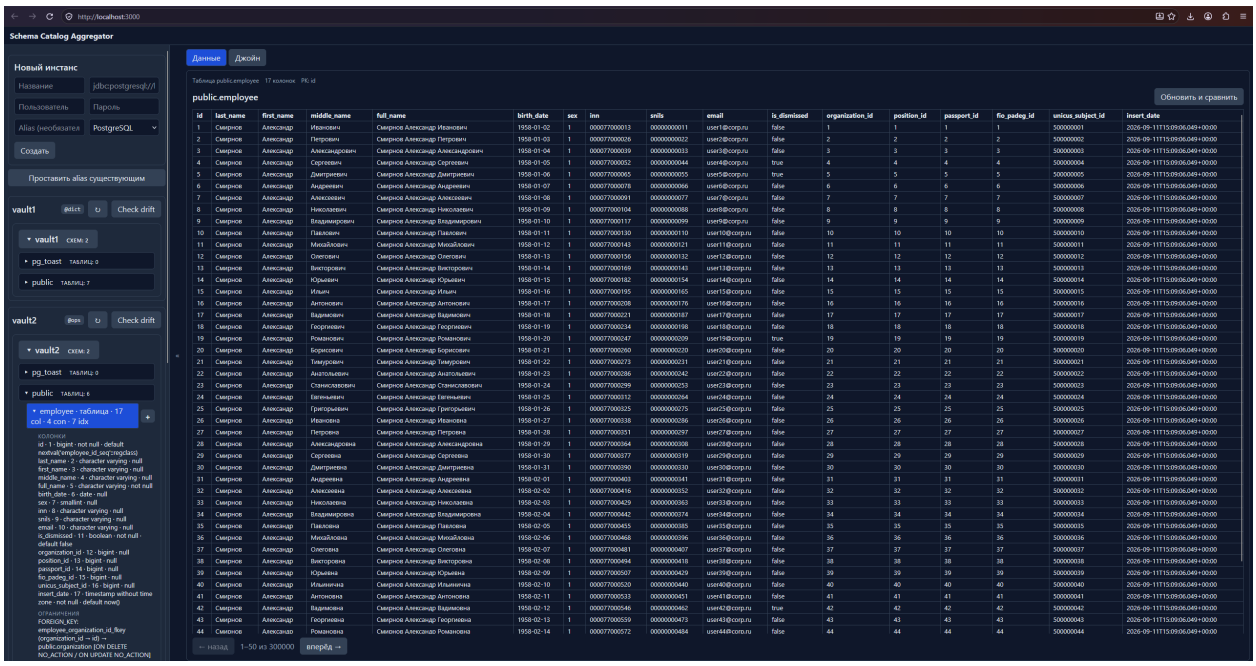


Рис. 13: Постраничный просмотр строк таблицы

Для контроля изменений данных используется построчное сравнение: клиент запоминает строки, отображавшиеся на экране, читает свежие строки источника и сравнивает их по первичному ключу. Добавленные, удалённые и изменённые строки подсвечиваются, а для изменённых показываются значения «было» и «стало». Результат приведён на рисунке 14.



Рис. 14: Построчное сравнение данных: эталон и источник

6.4 Сравнение схем

Сравнение схемы можно запустить из панели источника. Сервис собирает текущую схему источника и сравнивает её с сохранённым каталогом по именам и сигнатурам объектов, а не по внутренним идентификаторам. Результат открывается в выдвижной панели SchemaDiffDrawer. Добавленные, удалённые и изменённые объекты группируются по схемам и таблицам, поэтому можно быстро оценить масштаб расхождений.

6.5 Конструктор федеративных запросов

В режиме «Джойн» открывается визуальный конструктор (рисунок 15). Пользователь добавляет таблицы из разных источников, отмечает колонки результата и задаёт условия соединения. Конструктор не требует писать SQL вручную: он формирует структуру запроса и отправляет её в POST/api/join. Таблицы можно добавлять из дерева схемы кнопкой «+» или через поиск по полному имени alias.schema.table.

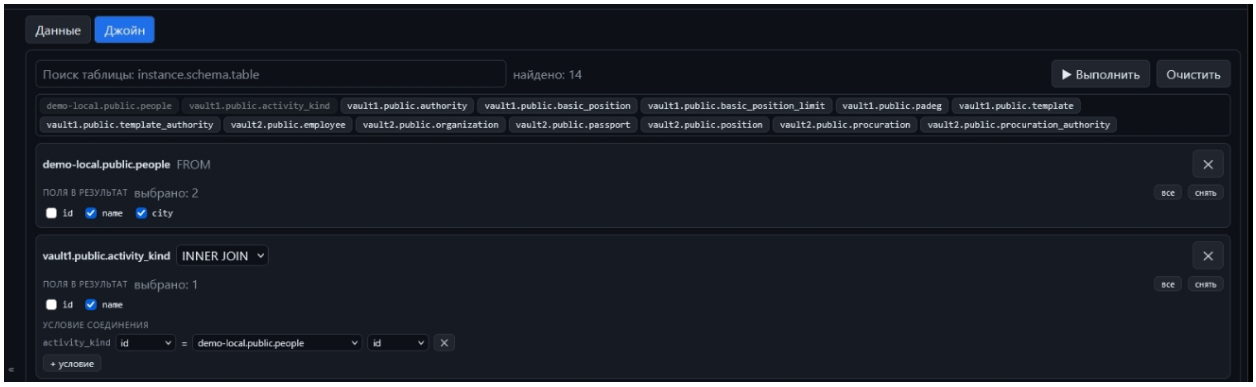


Рис. 15: Конструктор федеративного запроса

Результат запроса отображается в таблице с указанием количества строк и длительности выполнения (рисунок 16). Отдельная вкладка «История» показывает журнал выполненных запросов: SQL, статус, число строк и время выполнения (рисунок 17).

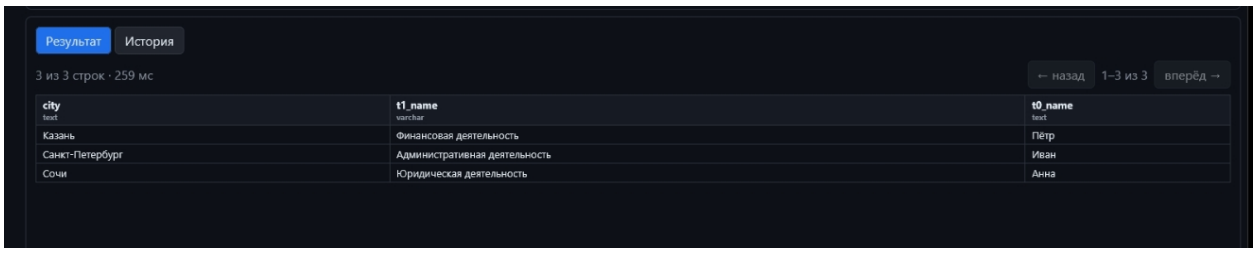


Рис. 16: Результат федеративного запроса

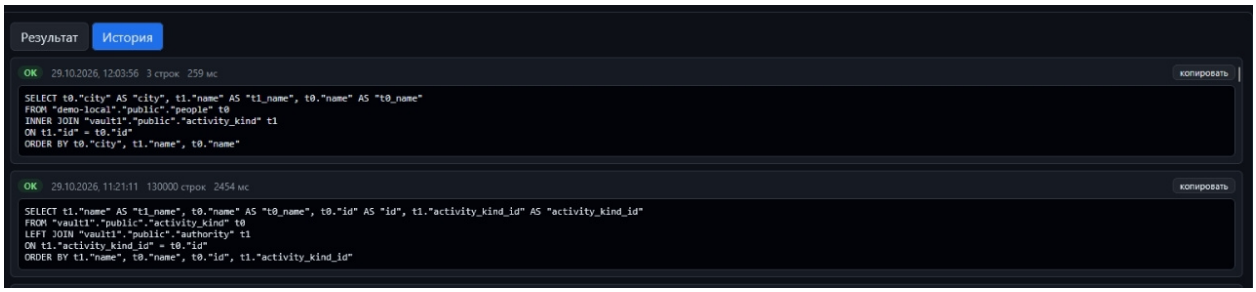


Рис. 17: История федеративных запросов